Fatima Jinnah Women University

Opening Portals of Excellence Through Higher Education

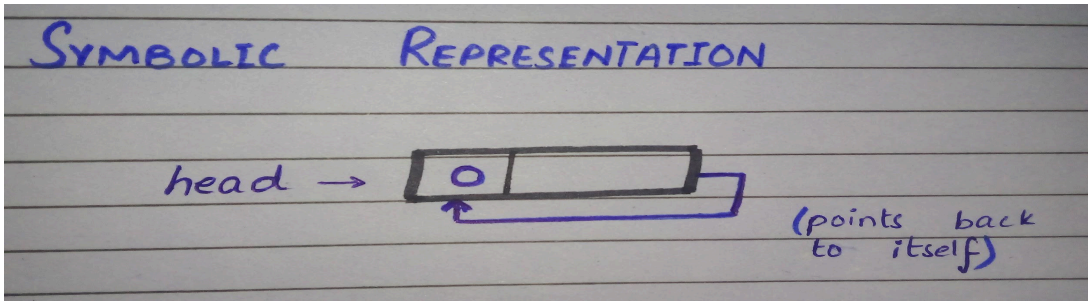
Data Structures And Algorithms

Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 15- Dec- 2025

LAB NO 9

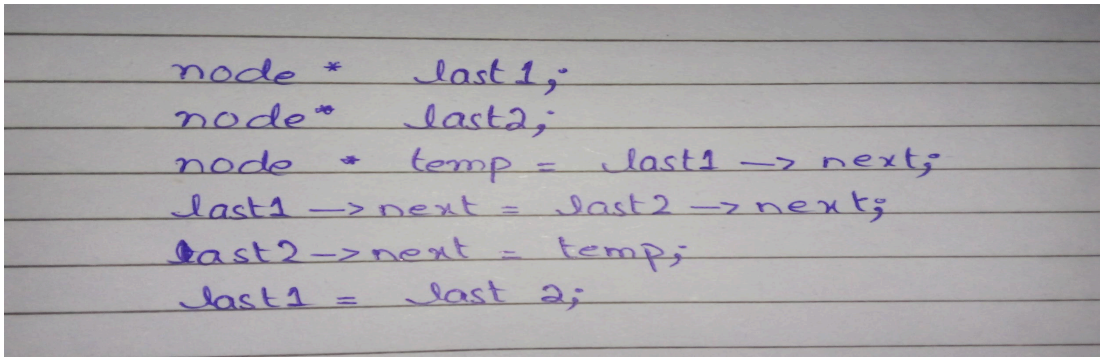
Task 1:

Draw a circular linked list of integers with a single node having a value 0.



Task 2:

Consider the following two circular linked lists with pointers on their last nodes.Write C++ statements to merge the two lists into one single list with POWERPOINT.



Code task 1

Implement the (class) Circular Linked List to create a list of integers. You need to provide the implementation of the member functions as described in the following.

```
class CList
{
private:
struct node
{int data;
node *next;
} *head;
public:
CList();
// Checks if the list is empty or not
bool emptyList();
// Inserts a new node with value ‘value’ after position
 ‘pos’in the list
void insert (int pos, int value);
// Inserts a new node at the start of the list
void insert_begin(int value);
// Inserts a new node at the end of the list
void insert_end(int value);
// Deletes a node from the beginning of the list
void delete_begin();
// Deletes a node from the end of the list
void delete_end();
// Displays the values stored in the list
void traverse();
};
```

Program:

```
#include <iostream>
using namespace std;

class CList
{
private:
   struct node
   {
      int data;
      node *next;
   } *head;

public:
```

```
CList()
{
   head = NULL;
}
bool emptyList()
{
   return (head == NULL);
}

void insert_begin(int value)
{
   node *newNode = new node;
   newNode->data = value;

   if (head == NULL)
   {
      head = newNode;
      newNode->next = head;
   }
   else
   {
      node *temp = head;
      while (temp->next != head)
         temp = temp->next;

      newNode->next = head;
      temp->next = newNode;
      head = newNode;
   }
}
void insert_end(int value)
{
   node *newNode = new node;
   newNode->data = value;

   if (head == NULL)
   {
      head = newNode;
      newNode->next = head;
   }
   else
   {
      node *temp = head;
      while (temp->next != head)
```

```
            temp = temp->next;

        temp->next = newNode;
        newNode->next = head;
    }
}
void insert(int pos, int value)
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    node *temp = head;
    for (int i = 0; i < pos; i++)
    {
        temp = temp->next;
        if (temp == head)
        {
            cout << "Invalid position\n";
            return;
        }
    }
    node *newNode = new node;
    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;
}
void delete_begin()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }
    if (head->next == head)
    {
        delete head;
        head = NULL;
    }
    else
    {
        node *temp = head;
```

```
        node *last = head;

        while (last->next != head)
            last = last->next;

        head = head->next;
        last->next = head;
        delete temp;
    }
}
void delete_end()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }
    if (head->next == head)
    {
        delete head;
        head = NULL;
    }
    else
    {
        node *temp = head;
        node *prev = NULL;

        while (temp->next != head)
        {
            prev = temp;
            temp = temp->next;
        }
        prev->next = head;
        delete temp;
    }
}
void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }
  node *temp = head;
```

```
        do
        {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != head);

        cout << endl;
    }
};

int main()
{
    CList cl;

    cl.insert_begin(10);
    cl.insert_end(20);
    cl.insert_end(30);
    cl.insert(1, 15);

    cout << "Circular List: ";
    cl.traverse();

    cl.delete_begin();
    cout << "After deleting beginning: ";
    cl.traverse();

    cl.delete_end();
    cout << "After deleting end: ";
    cl.traverse();

    return 0;
}
```

OUTPUT:

```
Circular List: 10 15 20 30
After deleting beginning: 15 20 30
After deleting end: 15 20
```